

Seguimiento de trayectoria a lazo cerrado basado en EKF para un robot omnidireccional

Maximiliano Fisz

1 Introducción

El objetivo de este trabajo es implementar los componentes necesarios para controlar un robot móvil de tipo omnidireccional, las diferencias de este proceso en relación a uno de tipo diferencial y exponer los resultados obtenidos en varios experimentos.

El trabajo sigue un enfoque iterativo, comenzando desde la funcionalidad más básica, accionar el movimiento del robot mediante un comando manual, hasta el seguimiento automático de trayectorias a lazo cerrado con *feedback* de *Extended Kalman Filter* (EKF).

Los fundamentos teóricos usados para las implementaciones son el paper "Kinematic Model of a Four Mecanum Wheeled Mobile Robot" y el material teórico visto durante la cursada.

En cada sección de este reporte encontraremos el desarrollo teórico respaldando la implementación del componente necesario para la funcionalidad, junto a las diferencias con el modelo diferencial (si las hubiera), los resultados de la experimentación y una guía para ejecutar algunos experimentos. Utilizaremos la imagen de Docker utilizada durante la cursada.

2 Modelo cinemático

2.1 Motivaciones

La plataforma omnidireccional con ruedas Mecanum es muy interesante, pero es una opción conveniente en casos muy específicos. La posibilidad de moverse de manera omnidireccional es muy útil en espacios restringidos donde no hay espacio para "virar" hacia el objetivo o donde no se desea rotar antes de trasladarse. El precio que se paga por esta ventaja en movilidad se paga en varias áreas: el costo *per se* del sistema de tracción es más costoso de obtener y mantener, la velocidad en trayectorias comparado con el modelo diferencial es bastante menor (cuando son posibles para este último) y las ruedas Mecanum son menos indulgentes sobre la superficie donde pueden operar sin perder tracción, requiriendo un control exigente sobre el espacio de operación.

2.2 Plataforma

El trabajo se realizará sobre el robot youBot (KUKA), una plataforma móvil existente con algunas modificaciones:

- Se ha intercambiado el manipulador y superficie de trabajo de la presentación original por un sensor LiDAR.
- El youBot cuenta con 4 ruedas de tipo Mecanum en sus extremos, la configuración de estas es de tipo "X", los *rollers* formando una cruz centrada sobre el centro del robot.

2.3 Objetivo

Traducir comandos de velocidad publicados a la red de ROS en movimiento del robot.

2.4 Cinemática

El paper nos proporciona las ecuaciones necesarias para la cinemática inversa que queremos implementar para poder controlar al robot mediante comandos de velocidades (v_x, v_y, ω) . La velocidad angular a aplicar en cada rueda se define en la ecuación 20 del paper con ω_i la i-esima rueda del robot:

$$\begin{aligned}\omega_1 &= \frac{1}{r} (v_x - v_y - (l_x + l_y) \omega) \\ \omega_2 &= \frac{1}{r} (v_x + v_y + (l_x + l_y) \omega) \\ \omega_3 &= \frac{1}{r} (v_x + v_y - (l_x + l_y) \omega) \\ \omega_4 &= \frac{1}{r} (v_x - v_y + (l_x + l_y) \omega)\end{aligned}$$

2.5 Odometría

Al igual que en el modelo diferencial, inicialmente, estimaremos la pose del robot media odometría, la evolución de las cuentas de los *encoders* que tenemos en cada rueda. El paper nos proporciona las ecuaciones (22, 23 y 24) para estimar las velocidades instantáneas del robot según las velocidades en cada rueda:

$$\begin{aligned}v_x(t) &= \frac{r}{4} (\omega_1 + \omega_2 + \omega_3 + \omega_4) \\ v_y(t) &= \frac{r}{4} (-\omega_1 + \omega_2 + \omega_3 - \omega_4) \\ \omega_z(t) &= \frac{r}{4(l_x + l_y)} (-\omega_1 + \omega_2 - \omega_3 + \omega_4)\end{aligned}$$

Integrando al igual que en el modelo diferencial, estimamos los cambios en la posición del robot entre cada lectura de los *encoders* y así obtenemos su pose.

2.6 Experimentación

Antes de comenzar a ejecutar los nodos, es necesario configurar el ambiente, el siguiente script agrega todas las referencias y librerías necesarias:

```
sim_files/copy.sh
```

Además, reutilizamos el proceso de compilación y *sourcing* usado durante la cursada cuando y donde sea necesario:

```
colcon build
source ./install/setup.sh
```

La implementación de la cinemática inversa y odometría forma parte del módulo "modelo omnidireccional". Para ejecutar el nodo, tenemos disponible un *script* que ejecuta este y el logger utilizado:

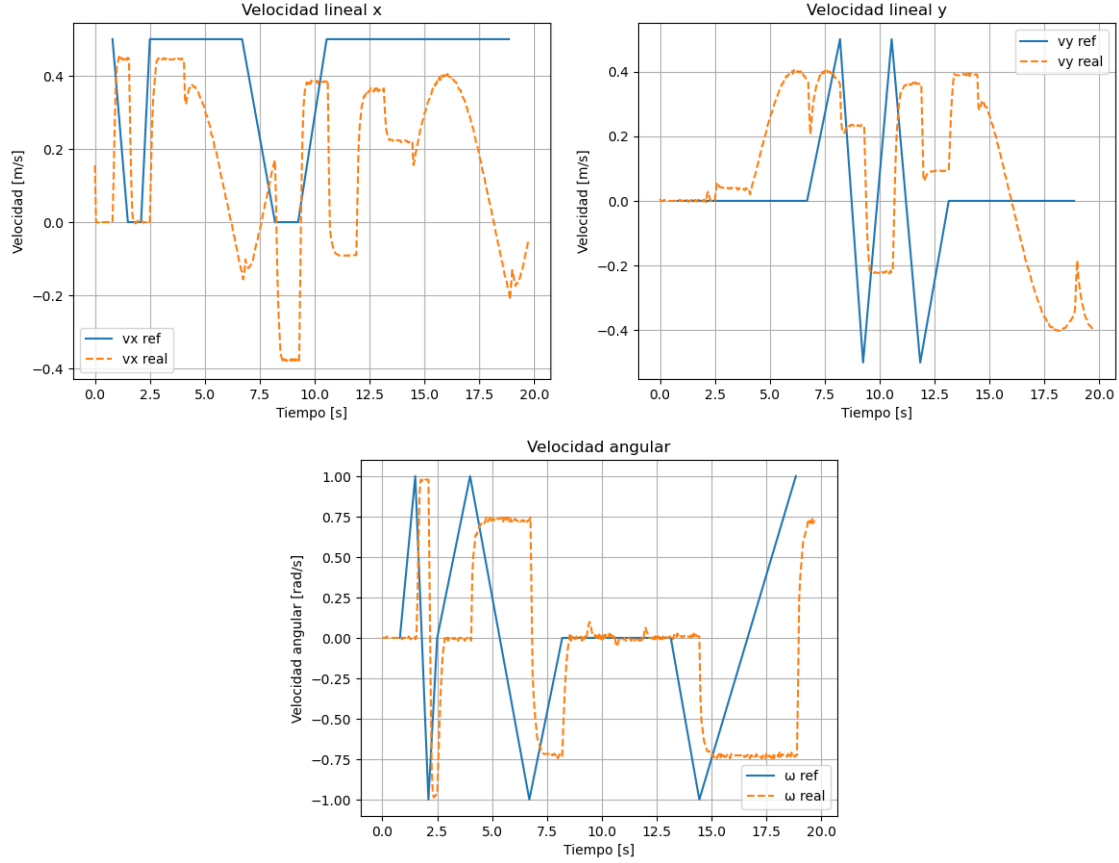
```
ros2 launch modelo_omnidireccional modelo_omnidireccional.launch.py
```

Para iniciar la simulación y visualizarla, podemos utilizar la escena provista de CoppeliaSim ("omni-ekf.ttt") junto a la configuración de Rviz "ej1.rviz".

Podemos enviar comandos de velocidad utilizando:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Ejecutando la simulación y enviándole diversos comandos de velocidad, comparamos los setpoints contra la estimación de velocidad por odometría.



El robot es controlable pero no tiene una respuesta muy rápida o precisa ante los comandos, particularmente cuando mezclamos velocidades lineales y angulares en un mismo movimiento. Al igual que con el modelo diferencial, el método acumula error por las imperfecciones del suelo, la respuesta no instantánea de los actuadores y otras variables que un modelo cinamático teórico no consideran; por lo que podemos notar el *drift* que se acumula entre la posición verdadera del robot y la calculada por odometría.

El código para la generación de los graficos se encuentra en:

```
utils/visualizador_velocidades.ipynb
```

3 Control a lazo cerrado con odometría

3.1 Objetivo

Definir trayectorias que el robot debe recorrer, estimando su pose con odometría para corregir el error entre los comandos de velocidad y el movimiento generado por estos. Las trayectorias a generar deben mantener al robot orientado opuesto al centro del espacio de simulación. Esto sera de utilidad en la proxima sección. El seguimiento de los waypoints se realiza mediante con un algoritmo *Pursuit Based*, usado también en el modelo diferencial.

3.2 Generación de trayectorias

Dado que nuestro objetivo incluye el seguimiento de trayectorias, reutilizamos los nodos existentes para la publicación y seguimiento de trayectorias del modelo diferencial, con la excepción del generador, que debe cambiar para poder aprovechar las nuevas capacidades de movimiento del robot.

Vamos a generar trayectorias suaves mediante splines con las siguientes restricciones:

$$x(t_i) = x_i, \quad x(t_f) = x_f, \quad \dot{x}(t_i) = \dot{x}_i, \quad \dot{x}(t_f) = \dot{x}_f$$

Entonces, $x(t)$ es un polinomio cúbico de la forma:

$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3$$

y su derivada:

$$\dot{x}(t) = a_1 + 2a_2t + 3a_3t^2$$

De esta forma, se pueden despejar los parámetros a_i :

$$a_0 = x_i$$

$$a_1 = \dot{x}_i$$

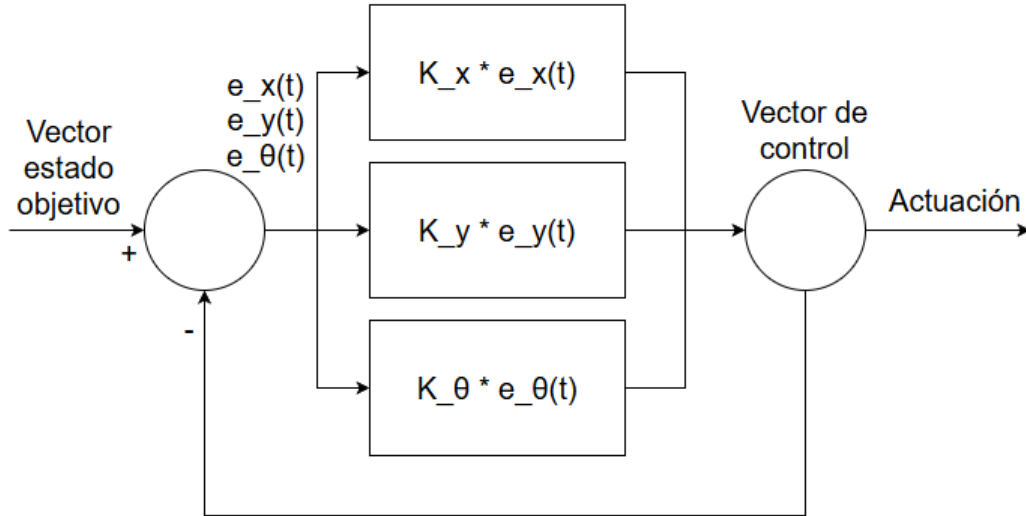
$$a_2 = \frac{3}{t_f^2}(x_f - x_i) - \frac{2}{t_f}\dot{x}_i - \frac{1}{t_f}\dot{x}_f$$

$$a_3 = -\frac{2}{t_f^3}(x_f - x_i) + \frac{1}{t_f^2}(\dot{x}_f - \dot{x}_i)$$

Repetimos esto mismo para $y(t)$ pero no para $\theta(t)$ ya que la orientación del robot la calcularemos durante la trayectoria para que mantenga la orientación opuesta al centro.

3.3 Control y seguimiento del objetivo

Se plantea un controlador Proporcional para cada uno de los grados de libertad del robot y se utiliza la odometría ya publicada para converger a las poses generadas por el generador de trayectorias.

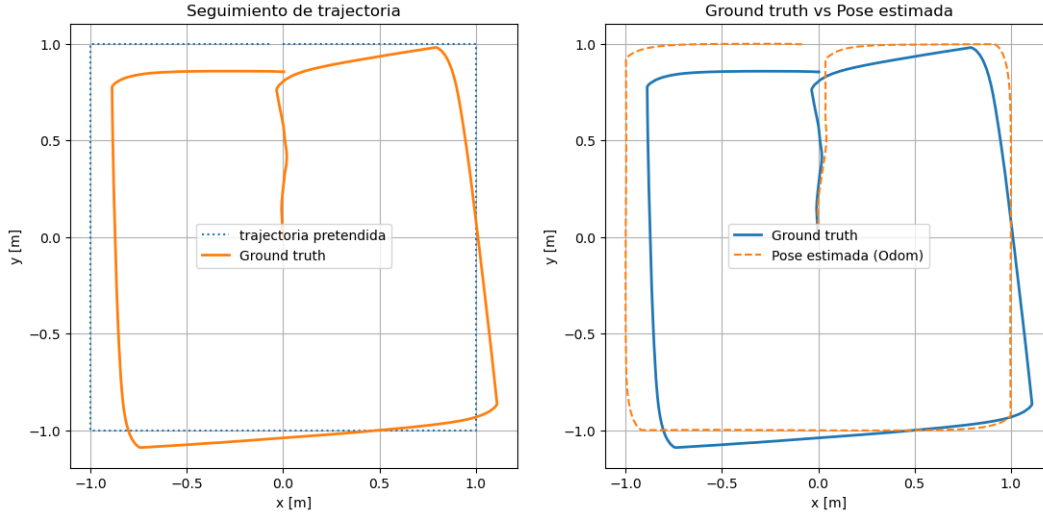


3.4 Experimentación

Ejecutamos la trayectoria propuesta: el cuadrado de 2 metros de lado desde el centro del espacio variando las constantes de ganancias y obtenemos los siguientes resultados:

K_X	K_Y	K_A	RMSE [m]
1.5	1.5	2.0	0.148
1.5	1.5	2.75	0.198
2.0	2.0	2.75	0.203
1.2	1.2	2.75	0.206
1.5	1.5	3.5	0.208

Nos quedamos con las constantes que minimizan el RMSE¹, métrica que usaremos para juzgar la estabilidad a lo largo del trabajo. Para este set de constantes visualizamos cuan bien predice la odometría la pose del robot durante la trayectoria comparada con la pose del simulador y como se refleja esta predicción en la trayectoria que ejecuta el robot en comparación con la pretendida



Podemos observar que el error acumulado y el feedback de la odometría generan *drifting* en la figura y su desviación a lo largo del recorrido. Se espera poder mejorar esta comparación sustancialmente más adelante cuando se reemplace la fuente de información del lazo cerrado.

La implementación de la nodos de trayectoria y control a lazo cerrado por odometría forman parte de los módulos "lazo abierto" y "lazo cerrado". Para ejecutar el nodo, tenemos disponible un *script* que ejecuta todo lo necesario para el seguimiento de la trayectoria:

```
ros2 launch lazo_cerrado lazo_cerrado.launch.py
```

Para iniciar la simulación y visualizarla, podemos utilizar la escena provista de CoppeliaSim ("omni-ekf.ttt") junto a la configuración de Rviz "ej2.rviz".

El código para la generación de los graficos se encuentra en:

```
utils/visualizador_trayectoria.ipynb
```

4 Control a lazo cerrado con EKF

4.1 Objetivo

Utilizar LiDAR, un mapa conocido y el filtro extendido de Kalman para estimar con mayor precisión la pose del robot a lo largo de la trayectoria, mejorando así la capacidad del lazo cerrado.

4.2 Sensado

Reutilizaremos el sensor LiDAR de la cursada pero con una diferencia: el mapa del espacio de trabajo no se construye de la primera lectura del sensor sino que es información conocida, en este caso: una lista de poses de los postes (provista por el simulador) que rodean el espacio de la trayectoria cuadrada que usamos como *benchmark*. Al ejecutar la trayectoria, referenciamos los postes detectados con el mapa y corregimos nuestra pose estimada. La implementación del nodo que controla y publica la información del LiDAR forma parte del módulo "laser".

¹Usamos $RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (e_{x,i}^2 + e_{y,i}^2)}$ considerando solo la posición del robot.

4.3 EKF

El modelo omnidireccional requiere de algunos ajustes para poder usar EKF correctamente:

El estado se mantiene:

$$\mathbf{x}_t = \begin{pmatrix} x \\ y \\ \theta \end{pmatrix}$$

Nuestro nuevo control permite movimiento individual en v_x , v_y y ω :

$$\mathbf{u}_t = \begin{pmatrix} v_x \\ v_y \\ \omega \end{pmatrix}$$

Nuestro modelo de predicción cambia:

$$\mathbf{f}(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w}) = \begin{pmatrix} x_{t-1} + v_x \Delta t \cos(\theta) - v_y \Delta t \sin(\theta) + w_x \\ y_{t-1} + v_x \Delta t \sin(\theta) + v_y \Delta t \cos(\theta) + w_y \\ \frac{\theta_{t-1} + \omega \Delta t}{\|\theta_{t-1} + \omega \Delta t\|} + w_\theta \end{pmatrix}$$

donde $w_{x/y/\theta} \sim \mathcal{N}(0, Q)$ es ruido blanco aditivo.

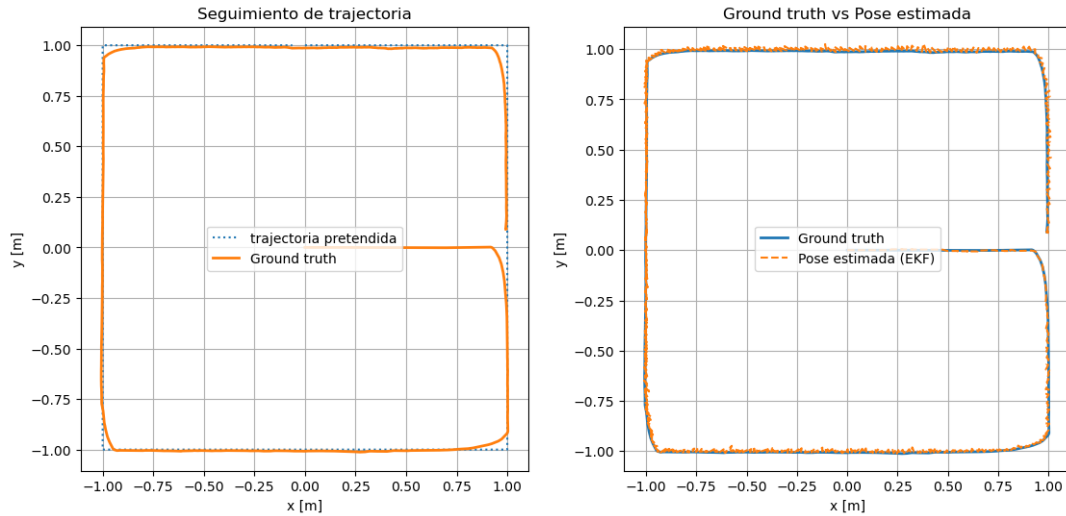
Recalculamos el jacobiano:

$$\mathbf{F}_t = \begin{pmatrix} 1 & 0 & -v_x \Delta t \sin(\theta) - v_y \Delta t \cos(\theta) \\ 0 & 1 & v_x \Delta t \cos(\theta) - v_y \Delta t \sin(\theta) \\ 0 & 0 & 1 \end{pmatrix}$$

4.4 Experimentación

Ejecutamos la trayectoria nuevamente pero esta vez usando la predicción de EKF de la pose del robot para guiarlo:

K_X	K_Y	K_A	RMSE [m]
1.5	1.5	2.0	0.015



La implementación de los nodos que utilizan EKF y el control a lazo cerrado con esta nueva información forman parte de los módulos "robmovil ekf" y "lazo cerrado ekf". Para EKF, se usa el módulo utilizado durante la cursada "kfilter". Para ejecutar el nodo, tenemos disponible un *script* que ejecuta todo lo necesario para el seguimiento de la trayectoria:

```
ros2 launch robmovil_ekf robmovil_ekf.launch.py
```

Para iniciar la simulación y visualizarla, podemos utilizar la escena provista de CoppeliaSim ("omni-ekf.ttt") junto a la configuración de Rviz "ej3.rviz".

5 Conclusiones

Los resultados son notablemente mejores cuando utilizamos EKF: la estimación de la pose es notablemente mejor en términos de RMSE y esto se refleja claramente en el seguimiento de la trayectoria del último experimento.

Debemos notar que estos experimentos se basan en una situación hipotética: tener un mapa conocido y perfecto al inicio de la operación. Esto no sucede así en aplicaciones no académicas pero no quita que una mejora sustancial también se podría encontrar al implementar fusión de sensado y EKF en la vida real. Sin esta ventaja artificial, probablemente se necesitaría refinar el modelo de referenciado de *landmarks* que actualmente es muy básico, las alternativas para esto dependen fuertemente de la situación particular. El otro *bottleneck* que queda es la generación de los waypoints para la trayectoria requerida. No es posible seguir la trayectoria cuadrada a la perfección únicamente con splines, debemos encontrar una implementación que combine estos con, por ejemplo, orientación sin traslación para lograr los giros de 90 grados requeridos, asumiendo que priorizamos el seguimiento perfecto sobre la velocidad.

6 Arquitectura

6.1 Organización del entregable

El entregable cuenta con los siguientes directorios y archivos, algunos ya mencionados previamente. Los directorios sin aclaración refieren a nodos requeridos para cada componente.

```
ROBMO/  
|-- coppeliaSim_files/ (archivos del simulador)  
|-- kfilter/  
|-- laser/  
|-- lazo_abierto/  
|-- lazo_cerrado/  
|-- lazo_cerrado_ekf/  
|-- modelo_omnidireccional/  
|-- robomovil_ekf/  
|-- robomovil_msgs/ (mensajes utilizados por el sistema)  
|-- rviz_files/ (archivos del visualizador)  
|-- sim_files/ (librerías y referencias externas)  
|-- utils/ (notebooks para visualización)  
|-- Dockerfile (imagen provista por la materia)  
|-- start-docker.sh  
|-- start-podman.sh
```

6.2 Tópicos

La arquitectura final del sistema tiene las siguientes publicaciones y suscripciones:

